

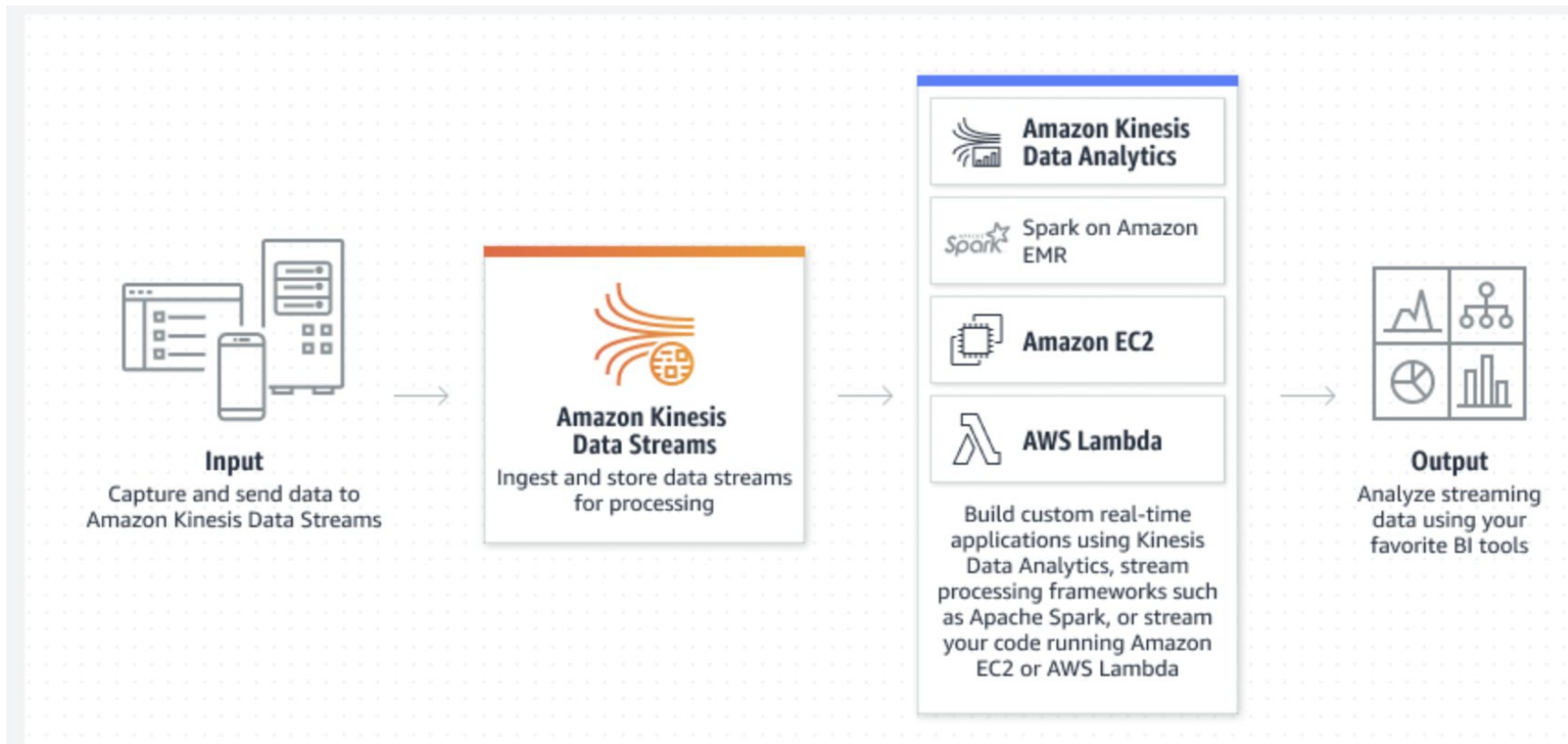
Amazon Kinesis makes it easy to collect, process, and analyze real-time, streaming data. It allows you to build applications that can continuously ingest and process large streams of data records in real-time.

Key Features:

- **Real-Time Processing:** Kinesis provides the capability to process and analyze data as soon as it arrives, which is critical for time-sensitive applications that need to make quick decisions based on the latest data.
- **Scalability:** Automatically scales to handle massive data throughput, without the need to manage infrastructure.
- **Data Collection from Various Sources:** Can collect data from countless sources such as website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events.

Components of Amazon Kinesis:

- **Kinesis Data Streams:** For building custom, real-time applications that process or analyze streaming data.
- **Kinesis Data Firehose:** For effortlessly loading streaming data into AWS data stores.
- **Kinesis Data Analytics:** For processing and analyzing streaming data with standard SQL.



- **Origin and Hosting**

- **Kinesis:** A managed service provided by AWS, offering seamless integration with other AWS services and a serverless option for handling data streams without managing infrastructure.
- **Kafka:** An open-source platform initially developed by LinkedIn and now a part of the Apache Software Foundation. Kafka can be self-hosted or managed by various cloud providers through services like Confluent.

- **Deployment and Management**

- **Kinesis:** Being a managed service, it significantly reduces the operational load of managing and scaling infrastructure. It's ideal for teams preferring cloud-native tools with minimal setup.
- **Kafka:** Requires more effort to set up, manage, and scale, especially in a self-hosted environment. However, it offers more control and flexibility over the system.

- **Performance and Scalability**

- **Kinesis:** Automatically scales to accommodate data throughput but has limits that can be increased upon request to AWS. Scaling involves increasing the number of shards in a stream.
- **Kafka:** Highly scalable and can handle high throughput with proper hardware and configuration. Scaling typically involves adding more brokers (servers) and properly partitioning topics.

- **Data Retention**

- **Kinesis:** Default data retention is 24 hours, extendable up to 365 days. Extended retention comes at an additional cost.
- **Kafka:** Configurable data retention policies based on time or size. It can retain data for as long as the disk space allows, supporting long-term storage if needed.

- **Pricing**

- **Kinesis:** Charges are based on the amount of data ingested, stored, and processed, along with the number of shards used.
- **Kafka:** For self-hosted Kafka, the main cost is infrastructure and operational efforts. Managed Kafka services charge based on usage, similar to Kinesis, but specific costs vary by provider.

- **Ecosystem and Integrations**

- **Kinesis:** Natively integrates with AWS services like AWS Lambda, Amazon S3, Amazon Redshift, and Amazon DynamoDB, offering a streamlined experience for AWS users.
- **Kafka:** Boasts a broad ecosystem and a large number of connectors for integrating with various databases, data lakes, and applications. This makes it highly versatile across different environments.

- **Use Cases**

- **Kinesis:** Ideal for AWS-centric organizations looking for a managed service for real-time analytics, log and event data collection, and application monitoring.
- **Kafka:** Suited for organizations requiring high throughput and scalability, complex real-time analytics, and a high degree of flexibility and control over their streaming data architecture.

- DynamoDB is a NoSQL database provided by Amazon Web Service (AWS). It can provide extremely high performance, more than 10 trillion requests per day with peaks greater than 20 million requests per second, and can support virtually any size with horizontal scaling. It is not uncommon for DynamoDB to serve over petabytes of data.
- From a 10,000 feet view, DynamoDB is basically a key-value store. It can be thought as a hash-map backed by some persistent storage. Two most important operations supported by DynamoDB are Get and Put.
- Unsurprisingly, the semantics of Get and Put operations are consistent with our understanding of hash-map or key-value store. For instance, we can use Put operation to persists a key-value pair to the DynamoDB and a subsequent Get operation will return the value we stored previously.
 - **Put("key", "value");**
 - **Get("Key") → Returns "value"**
- **DynamoDB in CAP Theorem:** DynamoDB is designed to provide high **availability** and **partition** tolerance. It achieves this by replicating data across multiple AWS regions and Availability Zones, ensuring that the system remains operational even in the event of server failures or network partitions. This design choice means that DynamoDB can continue to process reads and writes, maintaining the system's availability and partition tolerance.

Amazon DynamoDB's architecture is designed to provide fast, predictable performance at any scale for a wide variety of use cases, including mobile, web, gaming, ad tech, IoT, and many more. It's a fully managed, multi-region, durable database with built-in security, backup and restore, and in-memory caching. Here are the key components and architectural aspects of DynamoDB:

- **Core Components**

- **Tables:** The fundamental unit of data storage in DynamoDB. Tables contain items, and items contain attributes. Unlike traditional relational databases, there's no schema imposed on a table, allowing each item to have a different number of attributes.
- **Items:** Each item is a collection of attributes that is uniquely identifiable among all of the other items. In DynamoDB, there's no limit to the number of items you can store in a table.
- **Attributes:** Each item is composed of one or more attributes (name-value pairs), which are the fundamental data elements that DynamoDB stores. Attributes in DynamoDB are similar to fields or columns in other database systems.

- **Key Architectural Features**

- **Primary Key:** The primary key uniquely identifies each item in the table, allowing for quick access to data. DynamoDB supports two types of primary keys:
 - **Simple Primary Key:** Composed of a single attribute (Partition Key).
 - **Composite Primary Key:** Consists of two attributes (Partition Key and Sort Key).
- **Streams:** DynamoDB Streams capture a time-ordered sequence of item-level modifications in any DynamoDB table and store this information for up to 24 hours. Applications can access this stream and view, update, or delete items in a table.
- **DynamoDB Accelerator (DAX):** An in-memory cache for DynamoDB tables that delivers fast read performance for your tables at scale by reducing the response times from milliseconds to microseconds, even at millions of requests per second.
- **Partitions:** DynamoDB automatically spreads the data across multiple storage partitions for scalability and performance. Each partition is a self-contained unit of storage and computing.

Interview Questions

Q.) How does AWS Kinesis ensure data durability and what mechanisms are in place to protect against data loss?

Explanation: AWS Kinesis guarantees data durability by automatically replicating data across multiple Availability Zones in an AWS Region. Kinesis Streams store data for 24 hours by default, which can be extended up to 365 days, ensuring that data is not lost even in the event of infrastructure failures. Kinesis uses Synchronous replication to ensure that data is available in multiple locations as soon as it's written, providing high availability and data durability.

Q.) Describe the process of scaling a Kinesis stream and the impact it has on existing consumers.

Explanation: Scaling a Kinesis stream involves either increasing or decreasing the number of shards in the stream. This process is called resharding. Increasing the number of shards enhances the stream's ability to handle more data throughput. However, resharding can impact existing consumers as the partition key space is redistributed among the new set of shards. Consumers need to adapt to the new shard configuration to ensure continued data processing. AWS SDKs provide APIs to facilitate this adjustment, but it requires careful planning and implementation.

Q.) Explain how Kinesis Data Streams integrates with AWS Lambda and the use cases it enables.

Explanation: Kinesis Data Streams can directly trigger AWS Lambda functions to process data items as they arrive in the stream. This integration allows for real-time data processing without the need to manage polling or batch processing infrastructure. Use cases include real-time analytics, data transformation and enrichment, and immediate response to data-driven events, such as alerting and notifications based on specific data patterns detected in the stream.

Q.) Discuss the differences between Kinesis Data Streams and Kinesis Firehose and when you would choose one over the other.

Explanation: Kinesis Data Streams is a service for building custom, real-time applications that process or analyze streaming data. In contrast, Kinesis Firehose is a fully managed service designed for straightforward data delivery to AWS services like S3, Redshift, Elasticsearch, and Splunk without needing to write applications or manage

- **Key Architectural Features**

- **Primary Key:** The primary key uniquely identifies each item in the table, allowing for quick access to data. DynamoDB supports two types of primary keys:
 - **Simple Primary Key:** Composed of a single attribute (Partition Key).
 - **Composite Primary Key:** Consists of two attributes (Partition Key and Sort Key).
- **Streams:** DynamoDB Streams capture a time-ordered sequence of item-level modifications in any DynamoDB table and store this information for up to 24 hours. Applications can access this stream and view, update, or delete items in a table.
- **DynamoDB Accelerator (DAX):** An in-memory cache for DynamoDB tables that delivers fast read performance for your tables at scale by reducing the response times from milliseconds to microseconds, even at millions of requests per second.
- **Partitions:** DynamoDB automatically spreads the data across multiple storage partitions for scalability and performance. Each partition is a self-contained unit of storage and computing.

resources. You'd choose Data Streams when you need more control over data processing and consumer applications. In contrast, Firehose is suitable for scenarios where the goal is simply to deliver streaming data reliably to AWS destinations for analytics or storage.

Q.) How can you secure data being streamed through Kinesis Firehose, and what options are available for data encryption?

Explanation: Kinesis Firehose supports encryption of data in-flight using HTTPS and at rest using server-side encryption with AWS Key Management Service (KMS). For at-rest encryption, Firehose automatically encrypts data before storing it in the destination service and decrypts it when the data is read. You can use default AWS managed keys or customer-managed keys for more granular control. Additionally, IAM roles and policies can be used to control access to the Firehose stream and its data.

Q.) What mechanisms does Kinesis Firehose provide for transforming data before it reaches its destination, and how does error handling work in this context?

Explanation: Kinesis Firehose allows data transformation by integrating with AWS Lambda, where you can specify a Lambda function to transform incoming records before delivery. This setup supports use cases like data format conversion, data enrichment, and filtering. If the Lambda function encounters errors, Firehose provides two options: either skip the record or send the transformation failed records to a backup S3 bucket, depending on the configuration. This ensures that data is not lost and gives you the opportunity to debug and retry transformations.

Q.) How does DynamoDB achieve high availability and fault tolerance, and what role do global tables play?

Explanation: DynamoDB ensures high availability and fault tolerance by automatically replicating data across multiple servers in different Availability Zones within an AWS Region. For global availability and cross-region replication, DynamoDB offers global tables, which replicate your data automatically across your choice of AWS Regions. This setup enables fast, local reads/writes for globally distributed applications and adds another layer of fault tolerance by spreading the data across geographical locations.

Q.) Explain the concept of DynamoDB Streams and how it can be integrated with AWS Lambda for real-time processing.

Explanation: DynamoDB Streams capture time-ordered changes to items in a DynamoDB table, making the change data available for processing for up to 24 hours. Streams can trigger AWS Lambda functions, enabling real-time processing of changes (inserts, updates, deletes). This integration facilitates use cases such as maintaining materialized views, synchronizing data across systems, and implementing event-driven architectures by reacting to changes in DynamoDB data in real-time. Each record in the stream provides full details of a single data modification, allowing the Lambda function to process changes incrementally and efficiently. This setup is particularly useful for applications requiring immediate action upon data changes, such as sending notifications, updating search indexes, or aggregating metrics.

Q.) Describe how to use DynamoDB for handling time-series data, including considerations for table design and query performance.

Explanation: To handle time-series data in DynamoDB, design your table with a composite primary key where the partition key represents the entity identifier (e.g., device ID for IoT data) and the sort key represents the timestamp of the data point. This design allows efficient querying of time-series data for a given entity within a specific time range. To optimize query performance, consider pre-aggregating data at multiple granularities (e.g., minute, hour, day) in separate items or tables, depending on the access patterns. Additionally, use DynamoDB's TTL feature to automatically expire old data points to manage storage costs effectively. Be mindful of hot partitions and use techniques like salting the partition key with a random suffix for very high throughput use cases to ensure even distribution of write and read operations.

Q.) How can you optimize the cost and performance of a DynamoDB table experiencing variable read/write workloads?

Explanation: For DynamoDB tables with variable read/write workloads, consider using DynamoDB Auto Scaling to automatically adjust provisioned read and write capacity based on the actual application usage. This ensures that you pay only for the capacity you need while maintaining performance. For unpredictable or bursty workloads, consider using DynamoDB On-Demand Capacity, which offers flexible billing without capacity planning and automatically scales to accommodate workload changes. Additionally, leverage DynamoDB Accelerator (DAX) for read-intensive applications, which provides in-memory caching to reduce read latency and improve throughput. To further optimize costs, regularly review access patterns and adjust secondary indexes to eliminate those that are not used, as they incur additional storage and throughput costs.

Q.) Implementing efficient secondary index strategies in DynamoDB for complex query requirements.

Explanation: Secondary indexes in DynamoDB allow you to query data using alternate key attributes, which are not part of the primary key. There are two types of secondary indexes: Global Secondary Indexes (GSI) and Local Secondary Indexes (LSI). When designing secondary indexes, match them closely with your application's query patterns. GSIs offer more flexibility as they can have different partition and sort keys from the table and do not share throughput with the table. LSIs, on the other hand, must share the same partition key as the table but can have a different sort key. They share the table's throughput. Choose GSIs for querying across all items in a table and LSIs for querying within a partition. Consider the attributes included in the index to project only those necessary for queries to minimize storage costs. Always monitor the performance and adjust your indexing strategy as your application evolves and query patterns change.

Q.) Strategies for managing large items and attributes in DynamoDB to avoid performance degradation.

Explanation: DynamoDB has a maximum item size limit of 400 KB. To manage large items or attributes, consider splitting large attributes across multiple items or storing large attributes in Amazon S3 and keeping references (S3 object URLs) in DynamoDB. This approach reduces item size, leading to better performance for read and write operations. For attributes with large sets of data that might not be accessed together, store them in separate items linked by a common identifier. Use DynamoDB transactions to maintain consistency across related items. Additionally, compress large attribute values before storing them to reduce storage space and costs. Implementing these strategies helps maintain efficient data access patterns and ensures that your DynamoDB usage remains cost-effective and performant.

Q.) How would you design a real-time analytics pipeline for processing high-volume social media data using AWS Kinesis and DynamoDB?

Explanation: Start by ingesting social media data streams into Kinesis Data Streams, which can handle large volumes of data in real-time. Use Kinesis Data Analytics to apply SQL-based analytics or machine learning models to the data stream for real-time insights, such as sentiment analysis or trending topics. Store processed data in DynamoDB for fast, low-latency access. This setup allows for scalable, real-time analytics with durable storage for queryable insights.

Q.) Design a pipeline to collect and analyze IoT sensor data using AWS services. How would Kinesis Firehose and DynamoDB fit into this architecture?

Explanation: Ingest IoT sensor data directly into Kinesis Firehose, which can batch, compress, and encrypt the data before loading it into DynamoDB for storage. Use DynamoDB Streams to trigger AWS Lambda functions for further processing or aggregation of data as it arrives. This design minimizes infrastructure management and scales automatically with the influx of IoT data, providing a robust solution for IoT analytics.

Q.) How would you architect a data pipeline for ingesting and processing log files from multiple web servers in near-real-time with AWS Kinesis and DynamoDB?

Explanation: Utilize Kinesis Data Streams to collect log data from web servers. Implement log processing applications using Kinesis Client Library (KCL) or AWS Lambda to parse, filter, and transform the log data. Use Kinesis Firehose to load the processed data into DynamoDB, taking advantage of Firehose's automatic scaling and batching capabilities. This architecture ensures efficient log data processing and storage, allowing for real-time monitoring and analysis.

Q.) Explain how to design a scalable data ingestion system for a mobile gaming application using Kinesis and DynamoDB, focusing on player activity and in-game events.

Explanation: Use the Kinesis SDK within the gaming application to send player activity and in-game events to Kinesis Data Streams. Process and enrich the data in real-time using Kinesis Data Analytics or AWS Lambda functions, then route the data to DynamoDB for storage. Implement partition keys in DynamoDB based on player ID or game session to optimize query performance. This pipeline supports high-throughput data ingestion and enables real-time analytics on player behavior.

Q.) Describe a solution for processing and storing financial transactions in real-time using AWS Kinesis and DynamoDB, ensuring data integrity and low latency.

Explanation: Capture financial transactions as they occur using Kinesis Data Streams. Employ AWS Lambda to validate and enrich transactions in real-time, ensuring data integrity. Use Kinesis Firehose to batch and load the processed transactions into DynamoDB, taking advantage of DynamoDB's fast and consistent performance for transaction storage. Implement transactional conflict detection in your Lambda function

to handle potential duplicates or data inconsistencies, ensuring robustness in financial data processing.

Q.) How can AWS Kinesis, Kinesis Firehose, and DynamoDB be used to build a real-time recommendation engine for an e-commerce platform?

Explanation: Collect user activity data (clicks, views, purchases) via Kinesis Data Streams. Analyze the stream in real-time using Kinesis Data Analytics to identify patterns and generate recommendations based on user behavior. Use Kinesis Firehose to persist user activity logs and recommendation data to DynamoDB, where the recommendation engine can quickly access and serve personalized recommendations. This setup enables dynamic, real-time recommendations at scale.

Q.) Design a data pipeline for aggregating and analyzing marketing campaign data across various digital channels using AWS Kinesis and DynamoDB.

Explanation: Ingest raw campaign data from different digital channels into Kinesis Data Streams. Process the data in real-time with AWS Lambda to perform aggregation and analysis, such as click-through rates, engagement metrics, and conversion tracking. Store the aggregated results in DynamoDB for fast access by marketing dashboards and analytics tools. This approach provides a comprehensive, real-time view of campaign performance across channels.

Q.) Explain how to implement a system for monitoring and alerting on application health metrics using Kinesis, Kinesis Firehose, and DynamoDB.

Explanation: Collect application health metrics and logs via Kinesis Data Streams. Use AWS Lambda to filter, transform, and analyze the metrics in real-time, identifying anomalies or patterns indicative of application issues. Store historical metrics data in DynamoDB for trend analysis and root cause investigation. Configure Lambda to send alerts through Amazon SNS or another notification service when specific thresholds are breached, facilitating rapid response to application issues.

Q.) How would you design a pipeline for a video streaming platform to process and analyze viewer engagement data in real-time using AWS Kinesis and DynamoDB?

Explanation: Capture viewer engagement data (such as play, pause, and stop actions, watch time, and video quality metrics) from the video streaming platform using Kinesis Data Streams. This data is key for understanding viewer behavior and preferences. Process the data in real-time using Kinesis Data Analytics to compute engagement

metrics, such as average watch time, rebuffering events, and dropout rates. Use AWS Lambda functions triggered by the processed data in the stream to perform complex calculations or transformations if needed, and then load the results into DynamoDB. Kinesis Firehose can also be utilized to batch and load the raw engagement data into Amazon S3 for long-term storage and further batch processing. The real-time processed data stored in DynamoDB can be used to generate personalized content recommendations, improve the streaming quality based on viewer preferences, and identify popular content. Additionally, DynamoDB can serve dashboard applications to provide content creators and platform administrators with insights into viewer engagement trends.

Q.) Design a system for real-time fraud detection in financial transactions using AWS Kinesis, Lambda, and DynamoDB.

Explanation: Implement a multi-stage pipeline where financial transaction data is ingested into Kinesis Data Streams. Use AWS Lambda to subscribe to the stream and apply real-time fraud detection algorithms on each transaction. These algorithms can leverage machine learning models to score transactions based on their likelihood of being fraudulent. Store transactions and their fraud scores in DynamoDB, where they can be quickly accessed for blocking transactions or flagging them for review. DynamoDB's fast and predictable performance is crucial for ensuring that the fraud detection system does not introduce significant latency into the transaction processing pipeline. Additionally, use Kinesis Firehose to archive all transactions into Amazon S3, providing a dataset for training future machine learning models and auditing purposes.

Q.) Implementing a scalable log analytics solution for a distributed application using AWS Kinesis, Firehose, and DynamoDB.

Explanation: Collect log data from various components of a distributed application using Kinesis Data Streams. This centralized log collection is essential for monitoring the health and performance of the application across different environments and services. Use Kinesis Firehose to transform the logs (e.g., parsing, filtering) and load them into DynamoDB for real-time analysis and alerting. Complex log analysis can be performed using Lambda functions triggered by log data changes in DynamoDB Streams. This solution enables scalable and real-time log analytics, facilitating quick troubleshooting and operational insights.

Q.) Architect a scalable event tracking system for a mobile application using AWS Kinesis, Kinesis Firehose, and DynamoDB.

Explanation: Use the Kinesis SDK in the mobile application to send user event data (such as app opens, page views, and interactions) to Kinesis Data Streams. This real-time data ingestion allows for processing and analyzing user behavior as it happens. Employ Kinesis Firehose to batch, compress, and encrypt the data before loading it into DynamoDB, where it can be queried to gain insights into user engagement and application performance. This setup provides the foundation for a scalable event tracking system that can grow with the application, supporting millions of users and billions of events.

AWS Design Assignment - 1 (Design)

- **Problem Statement:** Designing a Real-time Data Ingestion Pipeline for a Flight Booking Application using AWS Services

- **Background:**

Air travel is one of the most frequently used modes of transportation globally. With the rise of technology, the majority of air ticket bookings have moved online. A hypothetical airline company, 'FlyHigh Airways', aims to modernize its flight booking application by implementing a real-time data ingestion pipeline to handle and process user bookings, cancellations, and search queries. This would not only allow the company to have real-time insights into their bookings but also offer a more personalized experience to their users based on their behaviors.

- **Requirements:**

- **Data Ingestion:** The system should be able to ingest real-time data from the flight booking application. This includes:
 - New bookings
 - Cancellations
 - Search queries made by users (destinations, dates, etc.)
- **Data Processing:** As the data is ingested in real-time, the system should be capable of processing this data for various use cases, such as:

- Real-time analytics
- Alerts on high demand routes
- Inventory updates based on bookings/cancellations
- **Storage:** The system should ensure data durability and be able to store large volumes of data without any significant delay.
- **Scalability:** As 'FlyHigh Airways' plans to expand its operations, the architecture should be scalable enough to accommodate the increasing number of users and data.
- **AWS Only:** The entire architecture should be based on AWS services.
- **Deliverables:**
 - **Architecture Diagram:** A detailed architecture diagram that showcases the real-time data ingestion pipeline using AWS services. Ensure every component in the architecture is labelled.
- **Assumptions:** List down any assumptions made while designing the architecture. This could include assumptions on data volume, rate of data ingestion, specific AWS service limits, or other constraints.
- **Scalability Justification:**
 - Provide a brief explanation on:
 - Why you believe your design is scalable.
 - How it handles peak loads or increased traffic.

- How it ensures minimal latency during real-time data processing.

- **Tips:**

- Think about the entry points for the data, i.e., where it originates from (mobile apps, web apps, etc.)
- Consider AWS services like Kinesis for real-time data streaming, Lambda for event-driven processing, and S3 or Redshift for storage.
- Scalability and fault-tolerance should be at the forefront of your design. Consider services that auto-scale and provide built-in redundancy.
- AWS offers various managed services. Leverage them to minimize operational overhead and focus on the application's business logic.

AWS Assignment - 2 (Implementation)

- **Objective:** Design and implement a real-time data processing pipeline in AWS to manage a simulated e-commerce platform's inventory data.
- **Problem Statement:**

Your e-commerce platform needs to track real-time inventory changes for its products. Every time a product is added, removed, or its quantity changes, an event is generated. Your task is to design a real-time processing system that will capture these events, process them, and update a real-time inventory system.
- **Requirements:**
 - **Data Source:** A mock script you'll design will simulate inventory events by generating and publishing them to an AWS Kinesis stream.
 - **Event Processing:** Use AWS EventBridge to capture events from the Kinesis stream and forward them for processing.
 - **Data Transformation and Storage:**
 - Process the real-time stream of data to calculate the latest inventory state for each product.
 - Store the processed data in DynamoDB, ensuring that inventory counts remain consistent and accurate.
 - **Inventory Events Include:**
 - Product added to inventory.
 - Product removed from inventory.

- Product quantity changes (increment or decrement).

- **Sample Event Data (Inventory Event):**

```
{
  "event_type": "product_added",
  "product": {
    "product_id": "P98765",
    "product_name": "Laptop",
    "quantity": 10,
    "timestamp": "2023-09-11T12:00:00Z"
  }
}
```

- **Steps:**

- Create a Kinesis stream to capture real-time inventory events.
- Design a mock script that simulates the inventory events and publishes them to the Kinesis stream in real-time.
- Set up AWS EventBridge to capture the events from the Kinesis stream.
- Process the real-time events, and according to the event type, update the product's state in DynamoDB.
- Ensure DynamoDB maintains the latest inventory state for all products and handles high-velocity updates reliably.

- **Deliverables:**

- **Code:**
 - Mock event generation script.

- Any required AWS Lambda functions or processing scripts.

- **Documentation:**

- A detailed flowchart of the data pipeline.
- Description of how events are processed and how DynamoDB maintains consistency.
- Challenges faced and the solutions implemented.

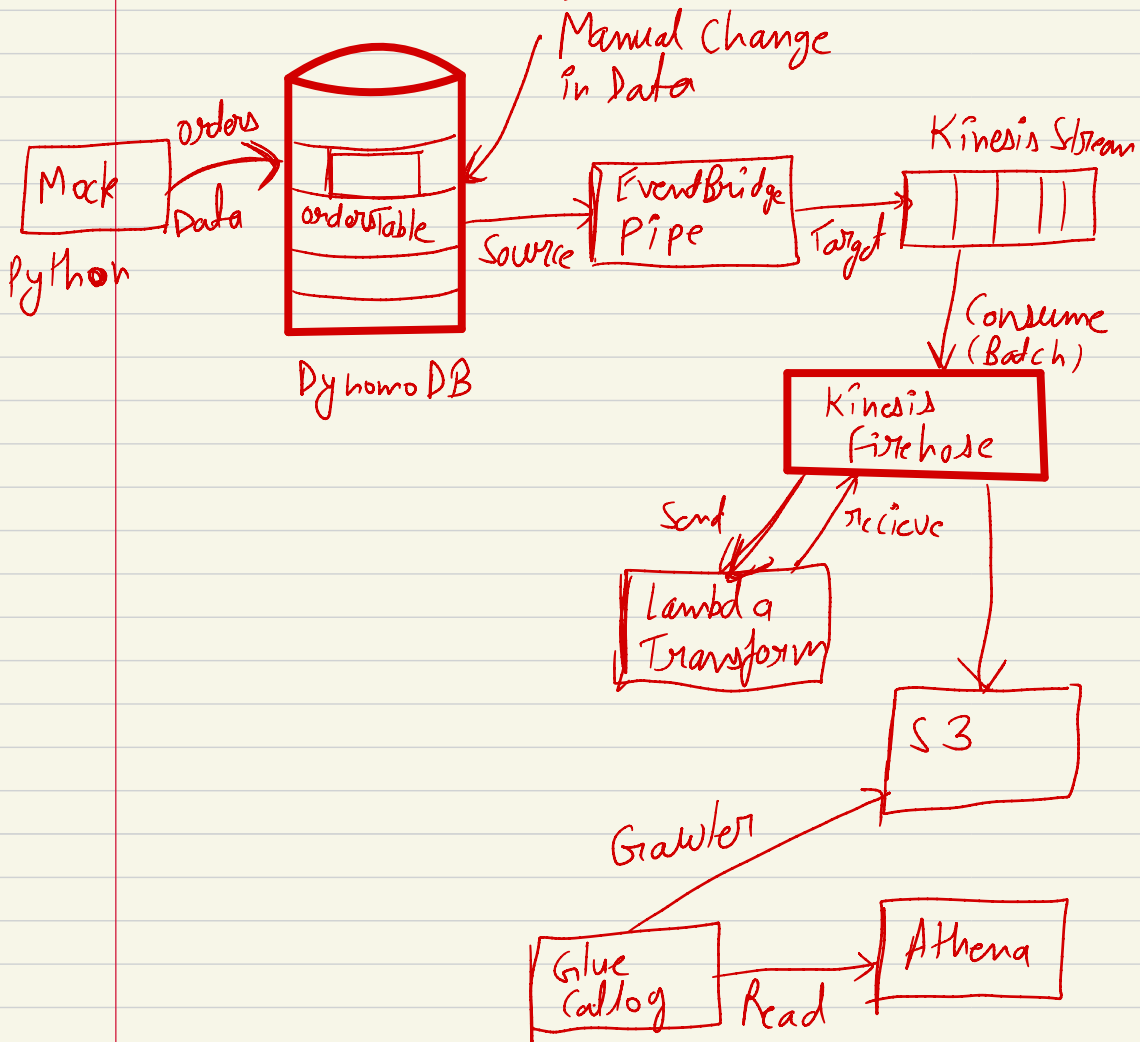
- **Output:**

- Demonstrate the real-time processing by showing the consistent state of inventory in
- DynamoDB after simulating a series of inventory events.



Project → (Change Data Capture)

⇒ CDC Data Pipeline for Sales Data Analysis



Test your knowledge : Class_Quiz

Quiz . 15 Total Points

😊 Great job! Keep pushing for that perfect score! 🚀🌟

Your marks : 12 / 15

1. What is Amazon Kinesis primarily used for?

- ☐ Web hosting
- ☒ Real-time data streaming
- ☐ Managed databases
- ☐ Virtual server management

✔ **Correct!**

2. Which Kinesis component allows you to process streaming data in real-time?

- ☐ Kinesis Firehose
- ☐ Kinesis Video Streams
- ☐ Kinesis Data Streams
- ☒ Kinesis Data Analytics

2. Which Kinesis component allows you to process streaming data in real-time?

- ☐ Kinesis Firehose
- ☐ Kinesis Video Streams
- ☐ Kinesis Data Streams
- ☒ Kinesis Data Analytics

✓ Correct!

3. How does Amazon Kinesis ensure data durability?

- ☐ By automatically saving data to an Amazon EC2 instance
- ☐ By using Amazon S3 for long-term storage
- ☒ By replicating data across multiple Availability Zones within a region
- ☐ By encrypting data using AWS KMS

✓ Correct!

4. Kinesis Data Streams can primarily be integrated with which of the following for data processing?

← ...

4. Kinesis Data Streams can primarily be integrated with which of the following for data processing?

- ☒ AWS Lambda
- ☐ Amazon EC2
- ☐ Both A and B
- ☐ Amazon S3

✗ **Incorrect!**

Both A and B is the correct answer

5. What is the maximum retention period for data stored in Kinesis Data Streams?

- ☐ 24 hours
- ☒ 7 days
- ☐ 365 days
- ☐ Indefinitely

✗ **Incorrect!**

365 days is the correct answer

6. What is the main function of Amazon Kinesis Firehose?

- ☐ To analyze streaming data
- ☒ To capture, transform, and load streaming data into AWS services
- ☐ To monitor streaming data
- ☐ To store streaming data

✓ Correct!

7. Which AWS service is NOT a direct destination for Amazon Kinesis Firehose?

- ☐ Amazon Redshift
- ☐ Amazon DynamoDB
- ☒ Amazon RDS
- ☐ Amazon Elasticsearch Service

✓ Correct!

8. What data format transformation does Kinesis Firehose support?

8. What data format transformation does Kinesis Firehose support?

- ☐ XML to JSON
- ☐ CSV to Parquet
- ☐ TXT to CSV
- ☒ All of the above

✗ **Incorrect!**

CSV to Parquet is the correct answer

9. How does Amazon Kinesis Firehose handle failed data delivery to destinations?

- ☒ Automatically retries delivery
- ☐ Deletes the data
- ☐ Sends an alert to Amazon CloudWatch
- ☐ Stores the data in an EC2 instance for manual review

✓ **Correct!**

10. Can Amazon Kinesis Firehose directly query data?

10. Can Amazon Kinesis Firehose directly query data?

☐ Yes

☒ No

✓ Correct!

11. What type of database is Amazon DynamoDB?

☐ Relational Database

☒ NoSQL Database

☐ Graph Database

☐ Time-Series Database

✓ Correct!

12. Which feature enables Amazon DynamoDB to automatically adjust its capacity to maintain performance?

☒ Auto Scaling

☐ Provisioned Throughput

☐ DynamoDB Accelerator (DAX)



12. Which feature enables Amazon DynamoDB to automatically adjust its capacity to maintain performance?

- ☒ Auto Scaling
- ☐ Provisioned Throughput
- ☐ DynamoDB Accelerator (DAX)
- ☐ Elastic Load Balancing

✓ Correct!

13. What is DynamoDB Streams used for?

- ☐ To monitor the performance of the database
- ☐ To automatically scale read and write throughput
- ☒ To capture changes to items in a DynamoDB table
- ☐ To stream video content

✓ Correct!

14. Which of the following is NOT a key component of DynamoDB?

- ☐ Table

14. Which of the following is NOT a key component of DynamoDB?

- ☐ Table
- ☒ Document
- ☐ Item
- ☐ Attribute

✓ Correct!

15. How does Amazon DynamoDB ensure data availability and durability?

- ☐ By automatically replicating data across multiple AWS Regions
- ☐ By storing data in Amazon S3
- ☐ By using Amazon EBS volumes for storage
- ☒ By automatically replicating data across multiple Availability Zones within an AWS Region

✓ Correct!